

Exercício Prático — Testes Manuais de API

Sistema: Banking API Training System

Objetivo: Praticar elaboração de casos de teste, execução manual, registro de bugs e análise de cobertura

Sobre Este Exercício

Você receberá a **especificação de requisitos** de uma API bancária simples e deverá:

1. Elaborar 10 casos de teste cobrindo diferentes funcionalidades

Este exercício simula um cenário real de trabalho em QA, onde você recebe uma especificação e precisa garantir a qualidade do sistema através de testes manuais.

Como Subir o Sistema

Pré-requisitos

Antes de começar, certifique-se de ter instalado:

- **Node.js** versão 18 ou superior (Download: <https://nodejs.org/>)
- Um navegador web (Chrome, Firefox ou Edge)
- Um cliente de API (escolha um):
 - **Postman** (Download: <https://www.postman.com/downloads/>)
 - **Insomnia** (Download: <https://insomnia.rest/download>)
 - Extensão **Thunder Client** no VS Code

Passo a Passo

1. Instalar as dependências do backend

Abra o terminal na pasta do projeto e execute:

```
cd backend  
npm install
```

Aguarde a instalação das dependências (express, cors, swagger, etc.).

2. Iniciar o servidor

Ainda no terminal, dentro da pasta backend, execute:

```
npm run dev
```

Você verá uma mensagem similar a:

Servidor rodando em <http://localhost:3000>
Swagger disponível em <http://localhost:3000/api-docs>

Pronto! A API está rodando.

3. Verificar se está funcionando

Abra o navegador e acesse:

- **API:** <http://localhost:3000>
- **Swagger (documentação interativa):** <http://localhost:3000/api-docs>

Se ambos carregarem, o sistema está pronto para os testes.

4. (Opcional) Abrir os frontends

Os frontends **não precisam de servidor** — basta abrir os arquivos HTML:

- **Frontend Web:** Abrir frontend-web/index.html no navegador
- **Frontend Mobile:** Abrir frontend-mobile/index.html no navegador

Especificação de Requisitos — Banking API

Visão Geral

O Banking API é um sistema bancário simplificado que permite:

- Criar, consultar, atualizar e deletar contas bancárias
- Realizar depósitos (adicionar dinheiro à conta)
- Realizar transferências entre contas

O sistema armazena dados **em memória** (ao reiniciar o servidor, os dados são apagados).

Requisitos Funcionais

RF-001: Criar Conta

Descrição: O sistema deve permitir a criação de uma nova conta bancária.

Endpoint: POST /accounts

Payload (JSON):

```
{
  "name": "João Silva",
  "email": "joao@email.com",
  "cpf": "111.111.111-11",
  "initialBalance": 500
}
```

Regras de Negócio:

- Todos os campos são obrigatórios
- initialBalance deve ser maior ou igual a 0 (não pode ser negativo)
- cpf deve ser único (não pode haver duas contas com o mesmo CPF)

Respostas Esperadas:

- **201 Created:** Conta criada com sucesso, retorna objeto completo com id, balance, createdAt
 - **400 Bad Request:** Campos ausentes, initialBalance negativo ou CPF duplicado
-

RF-002: Listar Todas as Contas

Descrição: O sistema deve retornar a lista de todas as contas cadastradas.

Endpoint: GET /accounts

Regras de Negócio:

- Retorna array vazio [] se não houver contas
- Retorna array de objetos conta se houver contas cadastradas

Respostas Esperadas:

- **200 OK:** Lista de contas (pode ser vazia)
-

RF-003: Buscar Conta por ID

Descrição: O sistema deve retornar os dados de uma conta específica pelo ID.

Endpoint: GET /accounts/{id}

Regras de Negócio:

- ID deve existir no sistema

Respostas Esperadas:

- **200 OK:** Retorna objeto da conta encontrada
 - **404 Not Found:** Conta não encontrada (ID inexistente)
-

RF-004: Atualizar Dados da Conta

Descrição: O sistema deve permitir atualizar nome e email de uma conta.

Endpoint: PUT /accounts/{id}

Payload (JSON):

```
{  
  "name": "João da Silva Junior",  
  "email": "joao.junior@email.com"  
}
```

Regras de Negócio:

- Pode atualizar name e/ou email (campos opcionais)
- **NÃO** pode alterar cpf nem balance

Respostas Esperadas:

- **200 OK:** Conta atualizada com sucesso
 - **404 Not Found:** Conta não encontrada
-

RF-005: Deletar Conta

Descrição: O sistema deve permitir remover uma conta do sistema.

Endpoint: DELETE /accounts/{id}

Regras de Negócio:

- Conta deve existir para ser removida

Respostas Esperadas:

- **200 OK:** { "message": "Conta removida com sucesso" }
 - **404 Not Found:** Conta não encontrada
-

RF-006: Realizar Depósito

Descrição: O sistema deve permitir adicionar dinheiro a uma conta.

Endpoint: POST /accounts/{id}/deposit

Payload (JSON):

```
{  
  "amount": 200  
}
```

Regras de Negócio:

- amount deve ser maior que 0 (não pode depositar valor zero ou negativo)
- Conta deve existir
- Saldo final = saldo atual + amount

Respostas Esperadas:

- **200 OK:** { "message": "Depósito realizado", "balance": 700 }
 - **400 Bad Request:** amount inválido (≤ 0)
 - **404 Not Found:** Conta não encontrada
-

RF-007: Realizar Transferência

Descrição: O sistema deve permitir transferir dinheiro entre duas contas.

Endpoint: POST /accounts/{id}/transfer

Payload (JSON):

```
{  
  "targetId": "uuid-conta-destino",  
  "amount": 100  
}
```

Regras de Negócio:

- Conta de origem (ID na URL) deve existir
- Conta de destino (targetId) deve existir
- amount deve ser maior que 0
- Conta origem deve ter saldo suficiente (saldo $\geq$ amount)
- Origem e destino **não podem ser a mesma conta**

Respostas Esperadas:

- **200 OK:** { "message": "Transferência realizada com sucesso", "sourceBalance": 400, "targetBalance": 300 }
 - **400 Bad Request:** Saldo insuficiente, amount inválido, ou origem = destino
 - **404 Not Found:** Conta origem ou destino não encontrada
-